Worksheet 11 — Memory-Safety & Exploitation (3 hrs)

> **Course:** Software Security (KOSEN69) · Week 11 **Aligned:** CWE-121 (stack-based buffer overflow) · CWE-787 (out-of-bounds write) · CWE-134 (externally-controlled format string) · CWE-242 (dangerous `gets`) **Signature game:** 💥 Fuzzing Race → Pwn the Binary
>
> **Ethics note:** The binaries built from `vuln.c` are **intentionally exploitable** and `win()` spawns `/bin/sh`. Build and run them **only inside the isolated toolbox container** (or a VM) — never on your host. Never run these on a machine you care about, and never aim these techniques at software you are not authorized to test.

Part 1 — Student Information

Name	Student ID	Date	Group
Sutapat Chucham	6531503085	24 Oct 2026	

AI Disclosure: Used an AI assistant (Claude) as a guided mentor throughout this worksheet — for hints/Socratic guidance on Parts 2–4, and as the subject of the required “Audit the AI” exercise in the format-fix prompt.

Part 2 — Lecture Questions

1. Draw the x86-64 stack frame for `parse_input()` in `vuln.c` (buf[64], saved RBP, saved return address). Where does `strcpy` start writing and which word must the exploit overwrite? **Ans:** strcpy starts writing at buf[0] and continues writing toward higher addresses. It encounters the saved RBP first (8 bytes), then the saved return address → total offset = 64 (buf) + 8 (RBP) = 72 bytes. You need to write junk for 72 bytes before reaching the return address that the exploit needs to overwrite.
2. Why is `gets()` (CWE-242) impossible to use safely, and why did C11 remove it from the standard library? **Ans:** gets() has no parameter to specify the "maximum buffer size" at all (unlike fgets(buf, sizeof(buf), stdin) which does) → whether it overflows or not depends 100% on the length of the input, which the person entering it (attacker) controls. No matter how careful the programr is, it can't be fixed because it's a design flaw of the API itself, not a human error that can be corrected with caution → C11 therefore removed it from the standard library altogether (not just deprecated it).
3. Explain CWE-134: why is `printf(user_input)` dangerous but `printf("%s", user_input)` safe? What can `%n` do? **Ans:** printf(user_input) is dangerous because the format string comes directly from the attacker. If you insert %x%x%x, printf will read values from the register/stack according to the specified number, even though no actual arguments are sent → garbage values that shouldn't be seen will be output (leak).

printf("%s", user_input) is safe because the format string is fixed (we control it) and there are actual arguments to pick from.

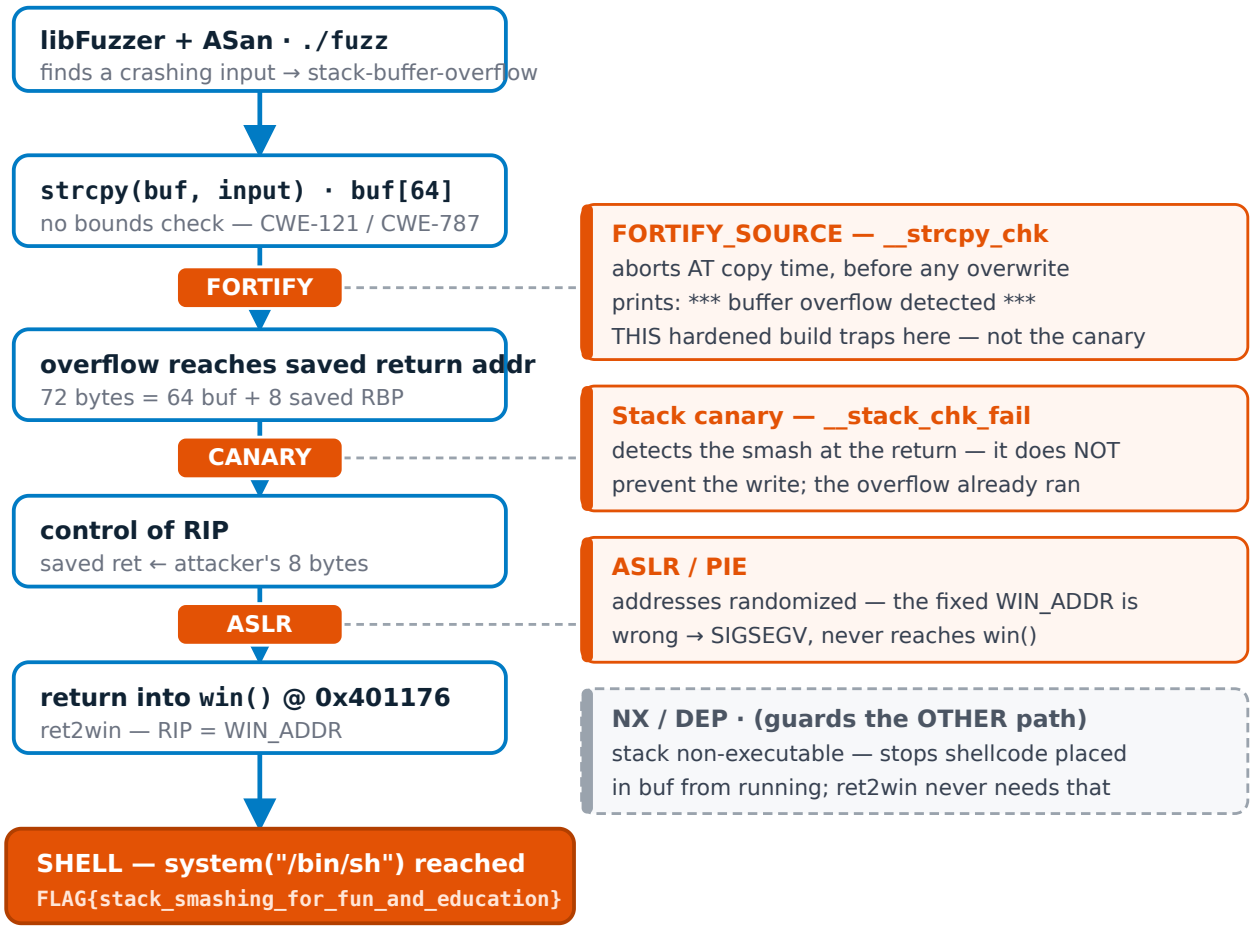
%n writes "the number of characters that printf has printed so far" to the address pointed to by the argument — when the attacker controls both the format string and the padding before %n → they can write any number to any address (arbitrary write primitive). 4. Name the four mitigations in the hardened build (`make vuln-`

`hardened`) — stack canary, PIE/ASLR, NX/DEP, FORTIFY — and say which one each defeats: ret2win, shellcode-on-stack, return-address overwrite detection.

Mitigation	
Stack canary:	return-address overwrite detection (Check when return)
PIE/ASLR:	ret2win (address always random → wrong hardcoded WIN_ADDR)
NX/DEP:	shellcode-on-stack (Aviod execute code on stack)
FORTIFY_SOURCE:	overwrite detection faster than canary (Intercept when strcpy is writing, even before the canary check.)

5. The CISA/ONCD roadmaps push memory-safe languages. Using `safe.rs`, explain why CWE-121 is *impossible* in safe Rust, not merely harder. **Ans:** String/&str are heap-backed, can grow on their own, and there is no fixed-size stack buffer to overflow. Even with a fixed array, Rust inserts bounds checks for every access → out of bounds = panic (controlled crash), not memory corruption. Most importantly: ownership + borrow checker checks at compile time code with the risk of unbounded write will not compile at all, meaning binaries with such bugs will never be created to run. This is different from C, where mitigation is just runtime detection (it compiles normally and relies on catching it during actual runtime).

Part 3 — Hands-on Lab (145 min)



Learning goals: Find the overflow with a coverage-guided fuzzer, exploit it (ret2win), watch mitigations break the exploit, then port the routine to safe Rust.

Prerequisites: the **toolbox container** (bundles `clang`+libFuzzer, `gdb`, `objdump`, `python3`) — Apple clang has no libFuzzer and `gdb` is painful on macOS, so do this week in the container. (`pip install pwntools` and `rustc` inside as needed; an isolated Linux VM also works.)

Environment setup

```
# from the repo root, build the toolbox once, then open a shell IN this
week's folder:
docker build -t softsec-toolbox labs/toolbox
docker run -it --rm --cap-add=SYS_PTRACE --security-opt seccomp=unconfined
\
    -v "$PWD/labs/week11-memory-safety-exploitation":/work -w /work
softsec-toolbox
# --- inside the container ---
make vuln                                          # teaching build:
```

```
mitigations OFF
make vuln-hardened                                  # mitigations ON
clang -g -fsanitize=address,fuzzer fuzz_harness.c -o fuzz   # libFuzzer +
ASan
setarch `uname -m` -R ./vuln                        # disable ASLR for
repeatable addresses
```

What to submit per task: the exact command(s) + crash/exploit output, a screenshot, and a 2–3 sentence mitigation note tied to the CWE.

Task 0 — Onboarding (15 min)

Run `make syntax` (parses both C files even on macOS) and confirm `make vuln` / `make vuln-hardened` build. Read the bug-class banner in `vuln.c`. **Deliverable:** build output + the list of CWEs `vuln.c` claims to demonstrate. CWE 'vuln.c' demonstrate: CWE-121 — Stack-based Buffer Overflow (strcpy into buf[64]) CWE-787 — Out-of-bounds Write (Buffer overflow) CWE-134 — Externally-Controlled Format String (printf(user_input)) CWE-242 — Use of Inherently Dangerous Function ('gets()' The one hidden in the comment before the gets() declaration)

Task 1 — Fuzzing Race: find the crash (35 min)

Goal: Make libFuzzer + AddressSanitizer crash on the `strcpy` overflow in `parse_input()` — CWE-121/787. **Steps:**

1. `./fuzz -runs=100000` (or just `./fuzz`).
2. Read the ASan report — note `stack-buffer-overflow` inside `parse_input` and the saved `crash-<hash>` file.
3. Replay it: `./fuzz crash-<hash>`. **Deliverable:** the ASan crash summary + the byte length of the crashing input + mitigation note. **ASan crash summary:** stack-buffer-overflow in strcpy at parse_input()

```
==477941==ERROR: AddressSanitizer: stack-buffer-overflow on address
0x7b64f5aca860
WRITE of size 69 at 0x7b64f5aca860 thread T0
    #0 in strcpy
    #1 in parse_input fuzz_harness.c:33:5
    #2 in LLVMFuzzerTestOneInput fuzz_harness.c:51:5

Address 0x7b64f5aca860 is located in stack of thread T0 at offset 96 in
frame
    #0 in parse_input fuzz_harness.c:31
  This frame has 1 object(s):
    [32, 96) 'buf' (line 32) <== Memory access at offset 96 overflows this
variable

SUMMARY: AddressSanitizer: stack-buffer-overflow in strcpy
```

the byte length of the crashing input (crash-2dd40706f754197dfc649fe05fb5e3d17908925c): 68 bytes **mitigation note:**This is a classic CWE-121/787 stack-buffer-overflow: a 68-byte input exceeds buf[64] and strcpy's unbounded copy writes past it into adjacent stack memory. In the hardened version, this strcpy would be replaced with __strcpy_chk which knows the target size and aborts the writing before it can corrupt the stack catching the bug even without ASan instrumentation, -D_FORTIFY_SOURCE=2 would do that.

Task 2 — Pwn the Binary: ret2win (40 min)

Goal: Overwrite the saved return address so the binary returns into `win()` and prints `FLAG{stack_smashing_for_fun_and_education}` — CWE-121. **Steps:**

1. Find `win()`'s address: `objdump -d vuln | grep '<win>:'` (or `gdb -batch -ex 'p win' ./vuln`).
2. Verify the offset with a cyclic pattern (do **not** trust 72 blindly): feed `cyclic(200)`, crash in gdb, `cyclic_find` the value in RIP. (On an Apple Silicon/arm64 Mac the toolbox has no platform pin, so it builds as an **aarch64** container — there is no `RIP`; use `info registers pc` / `p/x $pc` instead, and expect `SIGBUS`, not `SIGSEGV`. The offset itself is still 72 here.)
3. Edit `exploit_skeleton.py`: set `WIN_ADDR` to the address from step 1, confirm `OFFSET`.
4. `python3 exploit_skeleton.py` (pwntools path) — or `python3 exploit_skeleton.py | ./vuln`. (On arm64: the flag still prints, but the process then loops forever re-printing `[+] win() reached` instead of exiting — ARM64 keeps the return address in register `x30`, which still holds `win()`'s own address when it starts, so `win()`'s `ret` returns into itself. Expected; `Ctrl-C` once you've captured the flag.) **Deliverable:** the verified `OFFSET`, your `WIN_ADDR`, and the printed flag / `[+] win() reached` line.

|Verified OFFSET| 72 bytes (Prove using a cyclic pattern → saved RBP with offset 64 + 8)| |WIN_ADDR| 0x401196 (from objdump -d vuln | grep -A1 ':')| |Result| [+] win() reached — control flow hijacked! + flag FLAG{stack_smashing_for_fun_and_education}|

Task 3 — Hardened rebuild: watch the exploit break (25 min)

```
stack-frame
```

Goal: Observe modern mitigations defeating the same payload. **Steps:**

1. Run your Task 2 payload against `./vuln-hardened`.
2. Record the failure mode: `*** buffer overflow detected ***: terminated` (FORTIFY_SOURCE's `__strcpy_chk`, which on this build catches the overflow before the stack canary's epilogue check ever runs), or a SIGSEGV at a randomized address (PIE/ASLR makes `WIN_ADDR` wrong). **Deliverable:** the crash message + a note explaining *which* mitigation fired and why the fixed `WIN_ADDR` no longer works.

the crash message Enter your name: *** buffer overflow detected ***: terminated Aborted

NoteThe payload (72-byte padding + hardcoded WIN_ADDR) that worked against./vuln failed against./vuln-hardened with buffer overflow detected: terminated. This is FORTIFY_SOURCE's __strcpy_chk firing because -D_FORTIFY_SOURCE=2 -O2 substitutes the simple strcpy call with a bounds-checked version that

knows buf's real size (64 bytes) at compile time It aborts the writing since the 80-byte copy exceeds this before reaching the stack-canary check in the epilog or the ret instruction, when PIE/ASLR would have randomized win()'s real address anyway. In this build, all three defenses—FORTIFY, canary, and ASLR—are operational, but FORTIFY fires first and intercepts the flaw.

Task 4 — Rust rewrite: defend / fix it (30 min)

Goal: Show the bug class disappears in safe Rust using `safe.rs`. **Steps:**

1. `rustc safe.rs && ./safe`, then `python3 -c "print('A'*5000)" | ./safe` — note it never overflows or crashes.
2. Feed format-string metacharacters: `printf '%%x %%n' | ./safe` — note the program just reports the character count with no crash and no stack leak; the input is treated as ordinary data (CWE-134 impossible: `println!`'s format string must be a compile-time literal, so `println!(line)` would not even compile).
3. From the inline comments, write *why* `String`/`&str` + bounds checks make CWE-121/787 impossible. **Deliverable:** the safe output for both inputs + a 3–4 sentence explanation tying it to the CISA memory-safe roadmap.

```
$ python3 -c "print('A'*5000)" | ./safe
Hello, your input was 5000 characters long.

$ printf '%%x %%n' | ./safe
Hello, your input was 5 characters long.
```

Because String is heap-backed and grows automatically, even a 5000-byte input (vs. the 64-byte stack buffer that crashed vuln.c at 68 bytes) is handled properly with no fixed-size array to overflow, making CWE-121/787 impossible. Rust's println! macro requires a compile-time string literal for format string ("Hello, your input was {}..."). Because user input can only flow into a {} argument slot, %x %n is counted as 5 ordinary characters and not as format directives, making CWE-134 impossible. This explains why CISA/ONCD recommend memory-safe languages as a systemic risk reduction strategy: these two bug classes have been the main causes of exploitable vulnerabilities in C/C++ software for decades, and no amount of programr discipline has eliminated them. Rewriting in Rust requires porting legacy code and learning a new language, but it permanently closes the industry's oldest and largest attack surface by fixing the problem at the language/compiler level instead of relying on each programr to get every strcpy/printf call right.

Part 4 — Reflection

1. **Mapping:** table — bug | function in `vuln.c` | CWE | mitigation that stops it (canary / PIE / NX / Rust). |Bug|Function|CWE|Mitigation| |----|----|----|----| |Unbounded read via gets()| vulnerable()| CWE-242 (Use of Inherently Dangerous Function) | Only Rust (use read_to_string/read_line bound by a growable String instead) — canary/PIE/NX/FORTIFY do not directly fix this because gets() is still being called. | |Unbounded strcpy into fixed buffer | parse_input() | CWE-121 (Stack Buffer Overflow) + CWE-787 (Out-of-bounds Write) | FORTIFY_SOURCE (fastest, check at write) → If there is no FORTIFY, Stack canary catches it at the epilog → If it passes the canary (e.g., no canary at all), PIE/ASLR makes hardcoded WIN_ADDR incorrect → The real solution is to use Rust (no unbounded copy to overflow at all) | |Untrusted format string | format_bug() | CWE-134 (Externally-Controlled Format String) | None of

the four (canary/PIE/NX/FORTIFY) can protect against it! → It can only be fixed with Rust (forcing the format string to be a compile-time literal) or the correct C code (printf("%s", user_input)).|

2. **Real breach:** pick a memory-safety incident (Morris Worm fingerd overflow, or Heartbleed CVE-2014-0160). Which CWE applies, and would a stack canary or a Rust rewrite have prevented it? Heartbleed is an out-of-bounds heap read (CWE-125) — OpenSSL's server memcpy'd a response using an attacker-supplied length field without verifying it matched the actual received payload size, leaking up to 64KB of adjacent heap memory per request. A stack canary would not have prevented this, since canaries only detect a write that corrupts the value between a stack buffer and the saved return address — Heartbleed involved no stack write and no corrupted return address. A Rust rewrite would have prevented it structurally: bounds-checked slice indexing would trigger a safe panic! ("index out of bounds") instead of silently returning adjacent memory.

3. **Best mitigation:** compare "harden the C build" vs. "rewrite in Rust." Which gives durable risk reduction for new code, and what is the cost? Rust rewrites reduce risk more durablely than C build hardening. C mitigations include runtime detection on a language that allows unsafe operation, a continuous arms race (3 demonstrated FORTIFY halt our payload; Q2's Heartbleed proved that mitigations don't address all bug classes). Bug class elimination in Rust occurs during compile time, making it "impossible," not "harder." For new code, the cost is the team's learning curve and a younger ecosystem in some niches, a one-time expense against permanently addressing the industry's most persistent vulnerability class.

Grading rubric (100)

Criterion	Weight
Lecture questions (Part 2)	20
Exploitation + evidence (Tasks 1–3: fuzz crash, ret2win flag, hardened break)	40
Defense (Task 4: Rust rewrite + explanation)	25
Reflection (Part 4: mapping, breach, best mitigation)	15
Total	**100**

Evidence & Integrity (required)

- **Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` **in the same image as the evidence**. When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** — a cropped window carries nothing that identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted.
- **Personalized flag (if this lab issues one):** _____ *Flags are unique per student — submitting another student's flag is a violation. How to submit:* ***learn.zcr.ai/submit*** *(full guide: `SUBMISSION.md` in the repo root).*
- **Explain in your own words** *(graded on your reasoning, not copied text):*
    1. What did you do, and **why did the vulnerability work**? Fuzzing parse_input() with libFuzzer/ASan revealed a crash from a 68-byte input overflowing the 64-byte buf. A cyclic

pattern in gdb validated the exact offset to the saved return address: 72 bytes. I then created a payload of 72 bytes of padding followed by win()'s address and transmitted it to./vuln. Strcpy copies data into buf without a length check, so once the input exceeds 64 bytes, it keeps writing past the buffer into the saved RBP and then the saved return address next to it on the stack. Overwriting the return address with an address I chose sent the program's control flow to win() instead of vulnerable().

2. **Why does your fix actually stop it** — and what could still break it? My workaround replaces strcpy with snprintf(buf, sizeof(buf), "%s", src), which never writes more than 63 data bytes plus a guarantyd null terminator. The copy cannot reach past buf into the recorded RBP or return address, therefore the ret2win payload fails regardless of length. The fix only fixes parse_input()'s CWE-121 bug, vulnerable() still calls the equally dangerous gets(line) (CWE-242), and format_bug() still passes user input directly as a format string (CWE-134), neither of which snprintf touches.

🤖 Audit the AI (required)

AI is a power tool you must **distrust** — you are graded on your *critique*, not the AI's answer.

1. Ask an AI assistant to exploit **or** fix this week's vulnerability. Paste its full answer.

```
int parse_input(const char *src) {
    char buf[64];
    strncpy(buf, src, sizeof(buf));   // bounded copy — fixed!
    return (int)strlen(buf);
}
```

I replaced the unbounded strcpy() with strncpy(), which takes a maximum length parameter (sizeof(buf) = 64). This ensures the copy never writes more than 64 bytes into buf, eliminating the CWE-121 stack buffer overflow.

2. **Find what's wrong or risky** in it — insecure code, a subtly incomplete fix, a hallucinated API/function/CVE, a missed edge case, or wrong reasoning. Quote the exact line(s). Issue found: strncpy(buf, src, sizeof(buf)); strncpy does not null-terminate buf when src is ≥ 64 bytes long (according to the man page: “If there is no null byte among the first n bytes of src, the string placed in dest will not be null-terminated”). The next line, strlen(buf), then reads past the end of buf to find \0 in adjacent stack memory → changes from CWE-121 to CWE-125 (out-of-bounds read).
3. Produce the **correct, verified** version yourself and explain in 2–3 sentences why the AI's output was insufficient.

```
int parse_input(const char *src) {
    char buf[64];
    snprintf(buf, sizeof(buf), "%s", src);  // always null-terminates
safely
    return (int)strlen(buf);
}
```

Why the AI's output is insufficient: The AI constrained the copy to close the write-overflow (CWE-121), but it forgot that strncpy only guaranties a bounded write, not a null-terminated string, causing a new CWE-125 read-overflow in the next line. Because snprintf is documented to always null-terminate its output regardless of input length, with no edge case left unhandled, it fixes this "subtly incomplete fix" that this audit exercise asks students to catch.

> Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.

🧠 Comprehension & Prompt (required)

A. Explain in Plain English (EiPE). In 2–3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* — explain the mechanism, don't dump jargon. The program takes input from the user and copies it to memory without checking if the input is too long before copying. This causes it to overflow and overwrite the value that indicates where the program will return to continue working, which is located next to it in memory. This allows an attacker to choose where the program will jump to run the next part of the code.

B. Prompt Problem. Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the verified result**. *Graded on the prompt's precision and your verification — this trains problem decomposition and AI literacy (Denny et al. 2024).* Fix the CWE-121 stack buffer overflow in this C function:

```
int parse_input(const char *src) {
    char buf[64];
    strcpy(buf, src);
    return (int)strlen(buf);
}
```

Conditions:

1. No more than sizeof(buf) bytes, including the null terminator, may be written into buf by the repair.
2. Even if src is larger than or equal to sizeof(buf), the patch must ensure that buf is always null-terminated.
3. Describe the precise edge situation where a simple strncpy(buf, src, sizeof(buf)) fix would still be dangerous before providing the finished code.

Verified Result:

```
int parse_input(const char *src) {
    char buf[64];
    snprintf(buf, sizeof(buf), "%s", src);  // always null-terminates,
truncates safely
    return (int)strlen(buf);
}
```

Verification steps + expected outcome:

```
cp vuln.c vuln_fixed.c
# (Fix parse_input() In the file, please use the version above.)
gcc -fno-stack-protector -z execstack -no-pie -w -o vuln_fixed vuln_fixed.c
./vuln_fixed < payload.bin   # payload เดิมจาก Task 2 (72 bytes padding +
WIN_ADDR)
#Expected result: exploit should fail
```

In the first round, a broad prompt was used ("Fix the buffer overflow..."). Which allowed the AI to choose its own solution without anyone forcing it to consider edge cases regarding null-termination — this time, the prompt clearly specified two requirements (size bound + guarantyd null-termination) and pressured the AI to explain edge cases it might have missed before providing the code, making it impossible for the AI to avoid the issues we encountered in the Audit the AI section.